Linux Commands and Shell Scripting

A practical guide, with verified examples

- How this is organised
- Part 1 — The Command Line
  - 1.1 Navigation and files
  - 1.2 Viewing files
  - 1.3 grep — searching
  - 1.4 awk — column processing
  - 1.5 sed — stream editing
  - 1.6 sort, uniq, cut, tr
  - 1.7 Pipelines — the actual skill
  - 1.8 xargs
  - 1.9 Redirection
- Part 2 — Shell Scripting
  - 2.1 The header every script should have
  - 2.2 Variables and quoting
  - 2.3 Parameter expansion — replaces external commands
  - 2.4 Conditionals
  - 2.5 Loops
  - 2.6 Functions
  - 2.7 Exit codes and error handling
  - 2.8 A realistic script
- Part 3 — Systems and Debugging
  - 3.1 Processes
  - 3.2 What is this process doing?
  - 3.3 Performance
  - 3.4 Networking
  - 3.5 Files and text at scale
  - 3.6 Job control
  - 3.7 Shell productivity
  - 3.8 Environment
- Part 4 — Practice Path
  - Tools worth installing
- The ten things to actually remember

How this is organised

Part 1 is the **command line** — the tools you'll use daily. Part 2 is **shell scripting**. Part 3 is the **systems and debugging** side that matters for a low-latency role.

Every example was run and the output verified. The sample data is a trades CSV and an application log, because that's what you'll actually be grepping.

```
trades.csv:  timestamp,symbol,side,qty,price
app.log:     2026-08-03 09:15:45 WARN  Sequence gap detected seq=1042
```

Part 1 — The Command Line

1.1 Navigation and files

```
pwd                     # where am I
cd -                    # back to the PREVIOUS directory (underused)
ls -lhrt                # long, human sizes, reverse time -- newest last
ls -la                  # include hidden files

find . -name "*.cpp"                    # by name
find . -type f -mmin -60                # modified in the last hour
find . -size +100M                      # larger than 100MB
find . -name "*.log" -delete            # careful
find . -name "*.o" -exec rm {} +        # run a command per result

du -sh *                # size of each item here
du -sh . | sort -h      # human-readable sort
df -h                   # disk free
ncdu                    # interactive disk usage, if installed
```

`find -mmin -60` is the one to remember — "what changed in the last hour" answers most "why did it break" questions.

1.2 Viewing files

```
cat file                # whole file
less file               # pager: / to search, n for next, G to end, q to quit
head -20 file           # first 20 lines
tail -20 file           # last 20
tail -f app.log         # FOLLOW as it grows
tail -F app.log         # follow, and reopen if rotated
tail -n +100 file       # from line 100 onward
```

`tail -F` **not** `tail -f` for production logs — the capital F survives log rotation.

1.3 grep — searching

```
grep ERROR app.log                  # basic
grep -c ERROR app.log               # count matches -> 3
grep -i error app.log               # case insensitive
grep -v INFO app.log                # INVERT: everything except INFO
grep -E 'ERROR|WARN' app.log        # extended regex, alternation
grep -o 'id=[0-9]*' app.log         # print ONLY the matching part
grep -r "TODO" src/                 # recursive
grep -n ERROR app.log               # with line numbers
grep -A2 -B1 'Sequence gap' app.log # 2 lines after, 1 before
grep -w "id" file                   # whole word only
grep -l ERROR *.log                 # just list filenames that match
```

Verified output of `grep -o 'id=[0-9]*'`:

```
id=88
id=91
id=93
```

The four flags worth memorising: `-v` (invert), `-o` (only the match), `-c` (count), `-A/-B/-C` (context). Those cover most real use.

`grep -o` **is the one people don't know** — it turns grep from a filter into an extractor.

1.4 awk — column processing

awk splits each line into fields and runs your program on each one. `$1` is the first field, `$0` the whole line, `NR` the row

number, `NF` the field count.

```
awk -F, 'NR>1 {print $2, $4}' trades.csv        # skip header, print columns
awk -F, 'NR>1 {s+=$4} END {print "total:", s}'  # accumulate, print at end
awk '{print $3}' app.log                        # default separator is whitespace
awk 'NF' file                                   # print non-empty lines
awk 'length > 80' file                          # lines longer than 80 chars
```

Aggregating with an associative array — this is where awk earns its place:

```
awk -F, 'NR>1 {n[$2]+=$4*$5} END {for(s in n) printf "%-10s %.2f\n", s, n[s]}' trades.csv
```

```
INFY       450187.50
RELIANCE   1471075.00
TCS        477462.50
HDFCBANK   825000.00
```

Notional per symbol, in one line. Writing that in Python takes ten.

Filtering and formatting together:

```
awk -F, 'NR>1 && $3=="BUY" && $4>100 {printf "%-10s %5d @ %8.2f\n", $2,$4,$5}' trades.csv
```

```
RELIANCE     200 @  2451.00
INFY         150 @  1500.25
HDFCBANK     500 @  1650.00
```

Learn these five awk idioms and you're 90% covered:

Idiom	Does
`awk '{print $2}'`	extract a column
`awk '$3 > 100'`	filter by a condition
`awk '{s+=$1} END{print s}'`	sum
`awk '{n[$1]++} END{for(k in n) print k,n[k]}'`	group and count
`awk -F, -v OFS=,`	set input/output separators

1.5 sed — stream editing

```
sed 's/ERROR/CRITICAL/' app.log       # substitute FIRST match per line
sed 's/ERROR/CRITICAL/g' app.log      # ALL matches (g = global)
sed -i 's/old/new/g' file             # edit IN PLACE (no backup)
sed -i.bak 's/old/new/g' file         # in place, keeping file.bak
sed '/INFO/d' app.log                 # delete matching lines
sed -n '2,4p' app.log                 # print only lines 2-4
sed -n '$p' file                      # last line
sed '1d' file                         # drop the header
```

Capture groups — extract rather than replace:

```
sed -n 's/.*seq=\([0-9]*\).*/seq is \1/p' app.log
```

```
seq is 1042
```

`\( \)` captures, `\1` refers back. With `-E` you can drop the backslashes.

`sed -i` has no undo. Use `-i.bak` until you're confident, or test without `-i` first.

1.6 sort, uniq, cut, tr

```
sort file                   # lexical
sort -n file                # numeric
sort -rn file               # numeric, descending
sort -k2 file               # by 2nd field
sort -t, -k5 -rn trades.csv # comma-separated, 5th field, numeric desc
sort -u file                # sorted and deduplicated
sort -h                     # human sizes (1K, 2M, 3G)

uniq                        # dedupe ADJACENT lines -- usually needs sort first
uniq -c                     # prefix with a count
uniq -d                     # only duplicated lines

cut -d, -f2,4 trades.csv    # fields 2 and 4
cut -c1-10 file             # characters 1-10

tr 'a-z' 'A-Z' < file       # translate
tr -d '\r' < file           # delete characters (fixing CRLF)
tr -s ' '                   # squeeze repeats
tr '\n' ' '                 # newlines to spaces
```

`sort | uniq -c | sort -rn` **is the single most useful pipeline in Unix.** "Count occurrences, show the most common first":

```
awk -F, 'NR>1{print $2}' trades.csv | sort | uniq -c | sort -rn
```

```
3 RELIANCE
2 TCS
2 INFY
1 HDFCBANK
```

1.7 Pipelines — the actual skill

Individual commands are easy. **Composing them is the skill**, and it's what separates people who use the shell from people who fight it.

Extract error IDs, deduplicated and sorted:

```
grep ERROR app.log | grep -o 'id=[0-9]*' | cut -d= -f2 | sort -n | tr '\n' ' '
```

```
88 91 93
```

Count log levels:

```
awk '{print $3}' app.log | sort | uniq -c | sort -rn
```

```
3 INFO
3 ERROR
2 WARN
```

The mental model: each command reads lines, transforms them, writes lines. Build the pipeline one stage at a time, checking the output at each step. Don't write the whole thing then debug it.

1.8 xargs

Turns input lines into command arguments.

```
find . -name "*.csv" | xargs wc -l          # word count each file
echo "a b c" | xargs -n1 echo "item:"       # one argument per invocation
find . -name "*.o" | xargs rm               # delete them
find . -name "*.cpp" | xargs grep -l TODO   # which files contain TODO
```

Handle spaces in filenames properly:

```
find . -name "*.cpp" -print0 | xargs -0 grep TODO
```

Parallel execution — genuinely useful:

```
find . -name "*.cpp" | xargs -P 8 -n 1 ./compile.sh    # 8 at a time
```

1.9 Redirection

```
cmd > file          # stdout to file (overwrite)
cmd >> file         # append
cmd 2> errors.txt   # stderr only
cmd > out 2>&1      # both to the same file
cmd &> out          # shorthand for the above (bash)
cmd 2>/dev/null     # discard errors
cmd < input.txt     # stdin from a file

cmd | tee file      # to the screen AND a file
cmd | tee -a file   # appending

diff <(sort a.txt) <(sort b.txt)      # process substitution: treat output as a file
```

`2>&1` **order matters.** `cmd > out 2>&1` sends both to `out`. `cmd 2>&1 > out` sends stderr to the *terminal* and stdout to the file — because the redirections apply left to right.

`diff <(...) <(...)` is worth knowing — comparing the output of two commands without temp files.

Part 2 — Shell Scripting

2.1 The header every script should have

```
#!/bin/bash
set -euo pipefail
IFS=$'\n\t'
```

Flag	Effect
`-e`	exit immediately if any command fails
`-u`	error on an undefined variable
`-o pipefail`	a pipeline fails if **any** stage fails, not just the last

Without `pipefail`, `grep foo file \| wc -l` returns success even when grep found nothing — because `wc` succeeded. That silently hides failures.

`set -x` turns on tracing — prints each command before running it. The fastest way to debug a script.

2.2 Variables and quoting

```
name="Naman Kumar"
echo $name       # unquoted: word-splits into two arguments
echo "$name"     # quoted: one argument
```

Quote every variable expansion unless you specifically want splitting. Unquoted variables containing spaces are the single most common bash bug.

```
files=(a.txt "b file.txt" c.txt)
echo "${#files[@]}"       # 3        (length)
echo "${files[1]}"        # b file.txt
printf "%s\n" "${files[@]}"   # each element on its own line
```

`"${arr[@]}"` **with quotes** preserves elements containing spaces. `"${arr[*]}"` joins them into one string. Almost always you want `[@]`.

2.3 Parameter expansion — replaces external commands

These run in the shell, with no process fork. On a path `/home/user/data/trades.csv`:

Expansion	Result	Replaces
`${path##*/}`	`trades.csv`	`basename`
`${path%/*}`	`/home/user/data`	`dirname`
`${path%.*}`	`/home/user/data/trades`	strip extension
`${path##*.}`	`csv`	get extension
`${var:-default}`	value or `default`	fallback
`${var:=default}`	value, or set and use `default`	
`${var:?message}`	error out if unset	validation
`${s/-EQ/}`	replace first match	`sed`
`${s//old/new}`	replace all	`sed -g`
`${#s}`	length	
`${s:0:8}`	substring from 0, length 8	`cut`

The rule for `#` vs `%`: `#` strips from the **front**, `%` from the **back**. Doubled (`##`, `%%`) means greedy — longest match.

Mnemonic: on a keyboard, `#` is left of `%`.

2.4 Conditionals

```
[[ -f file ]]          # regular file exists
[[ -d dir ]]           # directory
[[ -x script ]]        # executable
[[ -z "$var" ]]        # empty string
[[ -n "$var" ]]        # non-empty
[[ "$a" == "$b" ]]     # string equality
[[ "$a" == Naman* ]]   # PATTERN match (no quotes on the pattern)
[[ "$a" =~ ^[0-9]+$ ]] # regex match
[[ -f a && -f b ]]     # and / or with && ||

(( n > 3 ))            # arithmetic -- use this for numbers
(( n++ ))
```

Use `[[ ]]` not `[ ]`. The double-bracket form is a bash builtin: no word splitting, no glob surprises, and it supports `&&`, `||`, `=~` and pattern matching.

Use `(( ))` for arithmetic — `[[ $a -gt $b ]]` works but `(( a > b ))` reads better and handles arithmetic properly.

2.5 Loops

```
for i in {1..10}; do echo "$i"; done
for ((i=0; i<10; i++)); do echo "$i"; done
for f in *.cpp; do echo "$f"; done
for f in "${files[@]}"; do echo "$f"; done

while read -r line; do echo "$line"; done < file
until (( n > 5 )); do ((n++)); done
```

Reading a file safely — this is the idiom to memorise:

```
while IFS=, read -r ts sym side qty px; do
    printf "%-10s %-6s %s\n" "$sym" "$side" "$qty"
done < trades.csv
```

- `IFS=,` sets the field separator for this read only
- `read -r` stops backslashes being interpreted
- Multiple variable names split the line into fields automatically

The classic trap: `cat file | while read ...` puts the loop in a **subshell**, so variables set inside are lost when it ends. Use `< file` redirection instead, or `while ... done < <(cmd)`.

2.6 Functions

```
count_errors() {
    local file="$1"                     # ALWAYS use local
    grep -c ERROR "$file" 2>/dev/null || echo 0
}

count=$(count_errors app.log)           # capture stdout
```

- `local` keeps variables scoped — without it everything is global
- `$1`, `$2` are arguments; `$@` is all of them; `$#` is the count
- Functions "return" via **stdout**, captured with `$(...)`
- `return N` sets the **exit code** (0–255), not a value

2.7 Exit codes and error handling

```
cmd
echo $?                     # exit code of the last command: 0 = success

cmd || echo "failed"        # run on failure
cmd && echo "ok"            # run on success
cmd || { echo "fatal"; exit 1; }

trap 'rm -f "$tmpfile"' EXIT       # cleanup on ANY exit, including errors
trap 'echo interrupted; exit 130' INT
```

`trap ... EXIT` **is the shell's RAII.** It runs whether the script finishes, errors out, or is killed — the right place for cleanup.

```
tmpfile=$(mktemp)
trap 'rm -f "$tmpfile"' EXIT
# ... use tmpfile, exit however you like, it gets removed
```

2.8 A realistic script

```bash
#!/bin/bash
set -euo pipefail

usage() { echo "usage: $0 -f FILE [-v]" >&2; exit 1; }

verbose=0
file=""
while getopts "f:v" opt; do
    case "$opt" in
        f) file="$OPTARG" ;;
        v) verbose=1 ;;
        *) usage ;;
    esac
done

[[ -z "$file" ]] && usage
[[ -f "$file" ]] || { echo "no such file: $file" >&2; exit 1; }

tmp=$(mktemp)
trap 'rm -f "$tmp"' EXIT

grep ERROR "$file" > "$tmp" || true       # || true: grep exits 1 on no match
count=$(wc -l < "$tmp")

(( verbose )) && echo "found $count errors"
exit 0
```

Two details worth noticing: `getopts` for proper argument parsing, and `|| true` after grep — because with `set -e`, a grep that finds nothing would kill the script.

Part 3 — Systems and Debugging

3.1 Processes

```bash
ps aux | grep myprog         # all processes
ps -ef --forest              # tree view
pgrep -f myprog              # just the PIDs
pidof myprog

top                          # live view
htop                         # nicer, if installed
kill -9 PID                  # SIGKILL (cannot be caught)
kill -TERM PID               # SIGTERM (polite, default)
pkill -f pattern
killall myprog

nice -n -20 ./prog           # higher priority (needs root)
taskset -c 2,3 ./prog        # PIN to cores 2 and 3
chrt -f 99 ./prog            # real-time FIFO scheduling
```

`taskset` **matters for your domain** — pinning a thread to a core keeps its cache warm and avoids migration jitter.

3.2 What is this process doing?

```
strace -p PID               # trace SYSCALLS
strace -c ./prog            # summary: which syscalls, how long
strace -e trace=network ./prog
ltrace ./prog               # trace LIBRARY calls
lsof -p PID                 # open files and sockets
cat /proc/PID/status        # memory, threads, state
cat /proc/PID/maps          # memory map
ls /proc/PID/task           # thread IDs
```

`strace -c` **is the one to reach for** when something is mysteriously slow — it shows immediately if you're making thousands of unexpected syscalls.

3.3 Performance

```
perf stat ./prog            # IPC, cache misses, branch misses
perf record ./prog; perf report
perf top                    # live profile
perf c2c record ./prog      # cache-line contention / false sharing

vmstat 1                    # system-wide, every second
iostat -x 1                 # disk
mpstat -P ALL 1             # per-CPU
sar -u 1                    # historical
free -h                     # memory
```

Three numbers from `perf stat` tell you the shape of a problem: low IPC means stalling, high LLC-load-misses means memory bound, high branch-misses means mispredict bound.

3.4 Networking

```
ss -tuln                    # listening sockets (replaces netstat)
ss -tp                      # with process names
netstat -i                  # interface stats -- check for drops
ip addr; ip route
tcpdump -i eth0 port 9000 -w capture.pcap
ethtool -S eth0             # NIC stats, including drops
ping / traceroute / mtr
```

`ethtool -S eth0 | grep -i drop` is the first thing to check when a feed handler is missing messages.

3.5 Files and text at scale

```
wc -l file                  # line count
split -l 1000000 big.csv    # split into chunks
zcat big.gz | head          # read compressed without extracting
zgrep ERROR app.log.gz      # grep compressed
tar czf out.tar.gz dir/     # compress
tar xzf in.tar.gz           # extract
rsync -avz src/ dest/       # sync, resumable
md5sum / sha256sum file     # checksums
```

3.6 Job control

```
./long_job &                # background
jobs                        # list
fg %1                       # foreground
bg %1
Ctrl-Z                      # suspend
nohup ./job &               # survives logout
disown -h %1                # detach from the shell

tmux new -s work            # terminal multiplexer -- survives disconnection
tmux attach -t work
screen -S work              # older alternative
```

Learn tmux. SSH into a box, start a long job in tmux, disconnect, come back tomorrow and it's still running with all its output.

3.7 Shell productivity

```
Ctrl-R          # search command history (the single best shortcut)
Ctrl-A / Ctrl-E # start / end of line
Ctrl-W          # delete previous word
Ctrl-U          # delete to start of line
Ctrl-L          # clear screen
Alt-.           # insert the LAST ARGUMENT of the previous command
!!              # repeat the last command
sudo !!         # repeat it with sudo
!$              # last argument of the previous command
cd -            # previous directory
```

`Ctrl-R` and `Alt-.` are the two that save the most time. Most people never learn `Alt-.` and retype paths constantly.

3.8 Environment

```
export VAR=value
echo $PATH
which cmd                   # where is it
type cmd                    # alias, function, builtin, or file?
env                         # all environment variables
source script.sh            # run in the CURRENT shell (so exports persist)
```

`~/.bashrc` for interactive shells, `~/.bash_profile` for login shells. Aliases and functions go in `.bashrc`.

Part 4 — Practice Path

Week 1 — daily driving. Stop using a GUI file manager. Navigate, search, and read logs entirely from the shell. Learn `Ctrl-R` and `Alt-.` on day one.

Week 2 — pipelines. Take a real log file and answer questions with one-liners: how many errors, which hour had most, which IDs failed. Build each pipeline one stage at a time.

Week 3 — awk and sed. Learn the five awk idioms in §1.4 and the sed substitution and capture forms. These two cover most text processing you'll ever need.

Week 4 — scripting. Write three real scripts: a log summariser, a build-and-test wrapper, and something that automates a task you currently do manually. Use `set -euo pipefail` in all of them.

Tools worth installing

```
ripgrep (rg)     # much faster grep, respects .gitignore
fd               # friendlier find
fzf              # fuzzy finder -- transforms Ctrl-R
bat              # cat with syntax highlighting
jq               # JSON processing
tmux             # terminal multiplexer
htop / ncdu      # process and disk browsers
shellcheck       # LINTS your scripts -- catches quoting bugs
```

`shellcheck` **is the highest-value one.** It catches unquoted variables, useless `cat`, and most of the subtle bugs, before you run the script.

The ten things to actually remember

1. `set -euo pipefail` at the top of every script
2. **Quote every variable expansion** — `"$var"`, always
3. `sort | uniq -c | sort -rn` — count and rank, the most useful pipeline there is
4. `grep -o` extracts; `grep -v` inverts
5. `awk '{n[$1]+=$2} END{for(k in n) print k,n[k]}'` — group and aggregate
6. `${path##*/}` and `${path%.*}` — no need to fork `basename`
7. `while IFS=, read -r a b c; do ... done < file` — the safe read idiom
8. `trap 'cleanup' EXIT` — the shell's RAII
9. `Ctrl-R` for history, `Alt-.` for the last argument
10. `tail -F` not `-f`; `[[ ]]` not `[ ]`; `(( ))` for arithmetic